

Calentasker

Fejlesztői dokumentáció



A vizsgaremeket készítették:

Fesető Imre 13.I

Kapus Kevin Kamill 13.D



Tartalom

1	Bevezetés	Error! Bookmark not defined.
2	Hitelesítés (Regisztráció és Bejelentkezés)	Error! Bookmark not defined.
3	Fő funkciók és alkalmazás-oldalak ...	Error! Bookmark not defined.
3.1	Feladatok (Tasks)	Error! Bookmark not defined.
3.2	Csoportok kezelése (Groups)	Error! Bookmark not defined.
3.3	Felhasználói Profil (Profile)	Error! Bookmark not defined.



1 Bevezetés

1.1 A dokumentum célja

A jelen fejlesztői dokumentáció célja, hogy átfogó képet adjon a Calentasker nevű alkalmazás technikai hátteréről, szerkezetéről és architektúrájáról. A dokumentum iránymutatást nyújt a projekt jövőbeli fejlesztői számára a környezet felépítéséhez, a kód értelmezéséhez és a rendszer továbbfejlesztéséhez.

1.2 A szoftver bemutatása

A Calentasker egy modern, webalapú központi platform a személyes és csoportos feladatok (teendők) kezelésére. Az alkalmazás célja, hogy kiváltsa a több különböző szoftver párhuzamos használatát, egyetlen, letisztult felületen kínálva megoldást a mindennapi és a csapatmunkában felmerülő feladatok rendszerezésére. Fő funkciói közé tartozik a regisztráció, profilkezelés, csoportok létrehozása, feladatok kiosztása, valamint a folyamatok nyomon követése és fájlok csatolása.

1.3 Alkalmazott technológiák és keretrendszerek

A projekt modern webes technológiákra épül:

- Backend: Python 3.13, Django 5.2, Django REST Framework (DRF), Pipenv, SQLite3 adatbázis, SMTP szolgáltatás.
- Frontend: Node.js (v20+), Vue.js 3 integrálva Vite keretrendszerrel, Bootstrap 5 a reszponzivitás biztosítására, Vue Router, Vue-i18n (lokalizáció), Axios.
- Fejlesztői eszközök és tesztelés: GitHub a verziókövetésre, Robot Framework és Selenium az End-to-End (E2E) teszteléshez, Postman az API hitelesítéshez és végpontok ellenőrzéséhez.



2 Rendszerarchitektúra

2.1 A rendszer felépítése

Az alkalmazás kliens-szerver architektúrát valósít meg. A szerver oldali logikát és az adatok perzisztens tárolását egy Django REST API biztosítja, mellyel a Vue.js alapú Single Page Application (SPA) aszinkron HTTP (Axios) hívásokon keresztül (JSON formátumban) kommunikál. A rendszerek közötti kommunikációhoz az azonosítást Token-alapú hitelesítés és CORS házirendek segítik.

2.2 Könyvtárszerkezet

A repository logikailag két fő ágra ("Frontend" és "Backend") oszlik, elválasztva a kliens és a szerver logikát. További kiegészítő elemekkel a projekt az alábbi struktúrát követi:

- Dokumentacio/: Dokumentációk, word dokumentumok, prezentáció.
- Ends/Backend/Calentasker: A Python/Django kód, amely magában foglalja a beállításokat (settings.py), a különböző alkalmazásokat (users, groups, tasks), a teszteket, és a média (media/attachments) könyvtárakat. Ugyanitt található az automatizált folyamatokat vezérlő Pipfile konfiguráció.
- Ends/Frontend/Calentasker: A Vue projekt struktúrája, benne a package.json definíciókkal, a formázást segítő fájlokkal, és az src mappával (Vue komponensek, útvonalak, store).



3 Fejlesztői környezet kialakítása

A rendszer futtatásához és fejlesztéséhez a frontend és backend környezet független és helyi előkészítése szükséges.

3.1 Rendszerkövetelmények

- Python 3.13
- Node.js (v20 vagy újabb ajánlott)
- Pipenv (globális Python csomagként telepítve: ``pip install pipenv``)
- Git verziókövető szoftver

3.2 A Backend konfigurálása és telepítése

1. Projekt megnyitása: Lépjen az ``Ends/Backend/`` mappába.
2. Függőségek telepítése: Futtassa a ``pipenv install`` parancsot a Pipfile-ban rögzített külső könyvtárak eléréséhez.
3. Virtuális környezet aktiválása: Futtassa a ``pipenv shell`` parancsot.
4. Django specifikus beállítások: Lépjen be a ``Calentasker`` almappába. Győződjön meg róla, hogy a ``.env`` fájl a megfelelő környezeti változókat (pl. SMTP e-mail beállítások, titkos kulcsok) tartalmazza. (Lásd az ``SMTP_SETUP.md`` specifikációt).
5. Adatbázis inicializálása:
 - Futtassa a ``python manage.py makemigrations`` parancsot
 - Futtassa a ``python manage.py migrate`` parancsot
6. Fejlesztői szerver indítása: Futtassa a ``python manage.py runserver`` műveletet. Az API alapértelmezetten a ``http://127.0.0.1:8000/`` porton válik elérhetővé.



3.3 A Frontend konfigurálása és telepítése

1. Projekt megnyitása: Navigáljon az `Ends/Frontend/Calentasker` könyvtárba.
2. Node modulok letöltése: Futtassa az `npm install` parancsot a Vue.js és a Vite szükséges függőségeinek felépítéséhez.
3. Fejlesztői szerver indítása: Futtassa az `npm run dev` parancsot.
4. Hozzáférés: A terminálban megjelenő URL (többnyire `http://localhost:5173/`) böngészőben való megnyitásával elérhetővé válik a grafikus felület és tesztelhető az összekapcsolt alkalmazás.



4 Adatmodell és adatbáziszerkezet

A backend a Django beépített relációs adatbázis-kezelő (ORM) rendszerét alkalmazza az SQLite3 motor felett. A rendszer tervezése a következő főbb entitásokat deklarálja:

4.1 Főbb entitások és kapcsolatok

- Felhasználók (Users): Felelős a felhasználók hitelesítéséért (Standard User model és kiegészítő adatok), jogosultságainak kezeléséért és az email megerősítés folyamatához rendelhető mezőkért.
- Csoportok (Groups): Több felhasználót összefogó entitás a közös feladatokhoz, tag-nyilvántartáshoz és hierarchia kialakításához (Many-To-Many kapcsolatokkal).
- Feladatok (Tasks): Tartalmazza a teendők leírását, határidejét, valamint állapotát (státuszát). Idegen kulcsokon keresztül egyedi személyhez és/vagy csoporthoz egyaránt dinamikusan hozzárendelhető.
- Mellékletek (Attachments): Az egyes feladatokhoz kötődő statikus állományok (adatfájlok, képek) nyilvántartása és metaadatainak biztosítása (a megfelelő Feladathoz FK-kapcsolattal rendelve).



5 API ÉS KOMMUNIKÁCIÓ

A szoftver elosztott működését a Standard RESTful alapelvekre és HTTP protokollra felírt rétegek teszik lehetővé. A kliens HTTP tranzakcióival (GET, POST, PUT, PATCH, DELETE metódusok) navigál és módosít a szerveren található értékeken.

A bemeneti és kimeneti JSON adatstruktúrák konvertálásáért (Serializing) és azok integritásellenőrzéséért a Django REST Framework (DRF) felel. Az API biztonságát token-alapú hitelesítési eljárással támogatja.

A backend mappában található ``Calentasker_API.postman_collection.json`` fájl átfogó formában tartalmazza a már kidolgozott, importálható és manuálisan tesztelhető összes végpont leírását.



6 Minősegbiztosítás és tesztelés

A szoftverkészítés során komoly hangsúlyt fordítottunk a megbízhatóságra:

- Backend Unit Tesztelés: Az egyes adatfeldolgozó egységeket (pl. modell kalkulációk, adatbázis-lekérdezések és hibrid JSON válaszok) Python-alapú belső unit tesztek vizsgálják a JSON Test Runner bevonásával. A futtatás manuálisan is ellenőrizhető a ``python manage.py test`` szkript, vagy egyéb fájlok útján (pl. ``test_api.py``).
- Frontend és E2E Tesztelés: Automatikus eseményszimuláció és böngészőautomatizálás alapozza meg a robusztus működést, amelyet külső keretrendszerek (Selenium, Robot Framework) végeznek el. Ezen e-2-e folyamatok célja, hogy a végponttól-végpontig megvizsgálják a regisztráció, belépés, csoportfejlesztés, valamint feladat-menedzsment felületeit.